

CSC343HY Practical 9

Functional Dependencies & Normalization

Every query you have written so far assumes the database schema is fixed and sensible. **Database design** asks the prior question: *is this schema even a good one?* A poorly designed table can store the same fact in many places at once, which invites update anomalies and contradictions. **Functional dependencies** (FDs) are the tool for reasoning precisely about which schemas are good, and **normal forms** are the yardsticks we measure them against. This practical builds up the machinery, FDs, Armstrong's axioms, and attribute closure, then applies it to classify and decompose schemas into 3NF and BCNF.

1. Functional Dependencies

Definition

A functional dependency $X \rightarrow Y$ (read “ X functionally determines Y ”) is a constraint on a relation: **any two tuples that agree on all of X must also agree on all of Y** . Here X and Y are *sets* of attributes.

$$X \rightarrow Y \quad \equiv \quad \text{for all tuples } t_1, t_2 : t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$$

An FD is a statement about *every legal instance* of the relation, not about the rows that happen to be present today. This has a sharp consequence, spelled out in the box below.

You can disprove an FD from one instance, but never prove one. If you find two rows that agree on X yet differ on Y , the FD $X \rightarrow Y$ is dead: it fails on this instance, so it cannot hold in general. But no matter how many rows satisfy $X \rightarrow Y$, the next insert might break it. FDs come from the *meaning* of the data (the real-world rules), not from staring at a snapshot.

Trivial vs. non-trivial

- **Trivial FD:** $X \rightarrow Y$ where $Y \subseteq X$. These always hold (e.g. `dish, chef` $\rightarrow$ `dish`) and carry no information.
- **Non-trivial FD:** $X \rightarrow Y$ where $Y \not\subseteq X$. These are the interesting constraints; they are what normal forms police.

Keys, in terms of FDs

Term	Meaning
Superkey	A set X with $X \rightarrow$ (all attributes); i.e. X determines the whole tuple.
Candidate key	A <i>minimal</i> superkey: no proper subset of it is still a superkey.
Prime attr.	An attribute that belongs to <i>some</i> candidate key.
Non-prime attr.	An attribute in <i>no</i> candidate key.

2. Armstrong's Axioms & Attribute Closure

The three axioms (and their consequences)

From a given set of FDs, others are logically *implied*. Armstrong's three axioms are **sound** (they only derive true FDs) and **complete** (they can derive every implied FD). Three more rules are handy shortcuts derivable from them.

Rule	Statement
Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$.
Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z .
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.
Union	If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
Decomposition	If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
Pseudo-trans.	If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$.

*Armstrong was **not** the astronaut. William W. Armstrong is a Canadian mathematician and computer scientist; the axioms are his.*

Attribute closure X^+

The closure X^+ is the set of *all* attributes determined by X under a set of FDs F . It is the workhorse of this entire practical: with it you can test superkeys, find candidate keys, check normal forms, and drive decompositions.

Algorithm. Start with $\text{result} := X$. Repeat until nothing changes: if there is an FD $V \rightarrow W$ in F with $V \subseteq \text{result}$, add W to result. When no rule fires, $X^+ = \text{result}$.

Two things closure tells you instantly. (1) $X \rightarrow Y$ holds (is implied by F) *if and only if* $Y \subseteq X^+$.
 (2) X is a superkey *if and only if* $X^+ = \text{all attributes}$.

Worked closure

Take schema (A, B, C, D, E) with $F = \{A \rightarrow BC, C \rightarrow B, D \rightarrow E, E \rightarrow D\}$.

Set	Closure	Why
A^+	ABC	$A \rightarrow BC$ adds B, C ; $C \rightarrow B$ adds nothing new.
B^+	B	no FD has B alone on the left.
C^+	BC	$C \rightarrow B$ adds B .
D^+	DE	$D \rightarrow E$ adds E ; $E \rightarrow D$ adds nothing new.
AD^+	$ABCDE$	$A \rightarrow BC$, then $D \rightarrow E$: everything.
AE^+	$ABCDE$	$A \rightarrow BC$, then $E \rightarrow D$: everything.

Since AD^+ and AE^+ are the full attribute set (and no proper subset of either is), **the candidate keys are AD and AE** . Hence the **prime** attributes are A, D, E and the **non-prime** attributes are B, C . We reuse this exact example in Section 4.

3. The Normal Forms

The forms are nested: each is stricter than the one before.

$$\text{BCNF} \subseteq \text{3NF} \subseteq \text{2NF} \subseteq \text{1NF}$$

A relation in BCNF is automatically in 3NF, and so on. “Higher” normal form = fewer redundancy anomalies, but as we will see, BCNF can cost you something 3NF keeps.

Form	Condition
1NF	Every attribute is atomic, no multi-valued or repeating fields.
2NF	In 1NF, and no <i>partial dependency</i> : no non-prime attribute depends on only <i>part</i> of a candidate key.
3NF	In 2NF, and no <i>transitive dependency</i> of a non-prime attribute. Formally: for every non-trivial $X \rightarrow A$, either X is a superkey <i>or</i> A is prime.
BCNF	For every non-trivial $X \rightarrow Y$, X is a superkey. (Strictly stronger: the “ A is prime” escape hatch of 3NF is gone.)

3NF vs. BCNF, the one difference that matters. Both demand a superkey on the left of every non-trivial FD, except 3NF forgives an FD whose right-hand side is a prime attribute. That single exception is why every relation can be decomposed into BCNF losslessly, but not always while **preserving all dependencies**; 3NF can always do both at once. You trade a little redundancy (3NF) for the ability to check every FD on a single table.

4. Worked Example: Decomposing into BCNF and 3NF

Work through this by hand alongside the handout. We use the schema and FDs from Section 2:

$$R(A, B, C, D, E), \quad F = \{ A \rightarrow BC, C \rightarrow B, D \rightarrow E, E \rightarrow D \}$$

Recall: candidate keys $\{AD, AE\}$; prime = A, D, E ; non-prime = B, C .

Part I: BCNF by repeated splitting

Idea. While some relation has a non-trivial FD $X \rightarrow Y$ whose X is *not* a superkey, split that relation into X^+ and $(X \cup (\text{rest}))$. Repeat until every FD everywhere has a superkey left-hand side.

Step 1. In R , the FD $A \rightarrow BC$ has $A^+ = ABC \neq ABCDE$, so A is not a superkey: **BCNF is violated**. Split on A^+ :

$$R_1(A, B, C) \quad R_2(A, D, E)$$

Step 2. Inside $R_1(A, B, C)$ the FD $C \rightarrow B$ has $C^+ = BC$, so C is not a superkey here: **violates BCNF**. Split $R_1()$ on $C^+ = BC$:

$$R_1(A, C) \quad R_2(C, B)$$

(Equivalently one could name the leftover piece (A, C) ; either way A and C end up together to keep the join lossless.)

Step 3. Inside $R_2(A, D, E)$ (the piece from Step 1, now call it $R_3(A, D, E)$) the FD $D \rightarrow E$ has $D^+ = DE$, so D is not a superkey here: **violates BCNF**. Split on $D^+ = DE$:

$$R_3(A, D) \quad R_4(D, E)$$

Final BCNF decomposition:

$$R_1(A, C), \quad R_2(C, B), \quad R_3(A, D), \quad R_4(D, E)$$

Every remaining FD ($A \rightarrow C, C \rightarrow B, D \rightarrow E, E \rightarrow D$) now sits in a table where its left side is a key, and each split was lossless because the shared attribute was a key of one side.

Part II: 3NF by synthesis

Idea. Compute a *minimal cover* of F , group FDs by their left-hand side into one relation each, and (crucially) make sure one relation contains a candidate key.

Step 1: minimal cover. Split right-hand sides to single attributes and drop anything redundant:

$$\{A \rightarrow B, A \rightarrow C, C \rightarrow B, D \rightarrow E, E \rightarrow D\}.$$

Here $A \rightarrow B$ is redundant because $A \rightarrow C$ and $C \rightarrow B$ already give it (transitivity). Removing it:

$$\text{minimal cover} = \{A \rightarrow C, C \rightarrow B, D \rightarrow E, E \rightarrow D\}.$$

Step 2: group by left side into relations:

$$R_1(A, C) \text{ (from } A \rightarrow C), \quad R_2(C, B) \text{ (from } C \rightarrow B), \quad R_3(D, E) \text{ (from } D \rightarrow E, E \rightarrow D).$$

Don't forget the key. None of $R_1(A, C), R_2(C, B), R_3(D, E)$ contains a full candidate key (AD or AE). To guarantee a lossless join, the synthesis algorithm adds one relation holding a candidate key, e.g. $R_4(A, D)$. Some course slides stop at the three grouped relations; for a strictly lossless 3NF design, include the key relation.

5. Exercises

Show your reasoning (closures, keys) not just the final answer.

1. Let $R(A, B, C, D)$ with $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D\}$.
 - (a) Compute B^+ and D^+ . Is B a superkey?
 - (b) Find *all* candidate keys of R .
 - (c) What is the highest normal form R satisfies? Justify.
2. Let $R(A, B, C)$ with $F = \{AB \rightarrow C, C \rightarrow A\}$.
 - (a) Find all candidate keys, and list the prime and non-prime attributes.
 - (b) What is the highest normal form R satisfies?
 - (c) Decompose R into BCNF. Is your decomposition dependency-preserving? What general point about 3NF vs. BCNF does this illustrate?

3. Find a minimal cover of $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C, AC \rightarrow D\}$. Show each stage: singleton right-hand sides, left-hand-side reduction, then redundancy removal.
4. Using *only* Reflexivity, Augmentation, and Transitivity, derive the Union rule: from $X \rightarrow Y$ and $X \rightarrow Z$, prove $X \rightarrow YZ$.
5. Explain, in a sentence or two, why a single populated instance of a relation can *disprove* a proposed FD but can never *prove* one.
6. A menu table `menu(dish, head_chef, signature_style)` has $F = \{\text{dish} \rightarrow \text{head_chef}, \text{head_chef} \rightarrow \text{signature_style}\}$ (each dish has one head chef; each chef has one signature style).
 - (a) What is the candidate key?
 - (b) Which normal form does it satisfy, and which does it violate? Name the offending dependency.
 - (c) Decompose it so the result is in BCNF.

6. Common Pitfalls

1. **Reading FDs off one instance.** A snapshot can only *break* an FD, never confirm it. FDs come from the domain's rules.
2. **Forgetting that keys are minimal.** A superkey determines everything; a candidate key is a superkey with nothing to spare. ADE can be a superkey while AD is the candidate key.
3. **Confusing 3NF's escape hatch with BCNF's.** 3NF forgives $X \rightarrow A$ when A is *prime*; BCNF forgives nothing but a superkey left-hand side. If you use " A is prime" to pass a relation, it is 3NF, not necessarily BCNF.
4. **Dropping the key relation in 3NF synthesis.** Grouping the minimal cover is not enough; add a relation containing a candidate key or the join is lossy.
5. **Splitting on the wrong side in BCNF.** When $X \rightarrow Y$ violates BCNF, split into X^+ and $X \cup (\text{rest})$. Keeping X in *both* halves is what makes the join lossless.
6. **Expecting BCNF to preserve every FD.** It may not. If dependency preservation matters, 3NF is the safe target.

Before You Leave

Checklist

1. I can state what $X \rightarrow Y$ means and explain why an instance can only disprove it.
2. I can compute an attribute closure X^+ and use it to test superkeys and find candidate keys.
3. I can classify a relation as 1NF / 2NF / 3NF / BCNF and name the dependency that blocks the next level.

4. I can decompose a schema into BCNF by repeated splitting, and into 3NF by synthesis from a minimal cover.
5. I can explain the 3NF-vs-BCNF trade-off (dependency preservation vs. redundancy).